

Question 1: What did you have to change in each version to add the second moving average, and where? Paste in the changed lines from both.

In `demo_modified.py`, I added a second slider widget directly in the sidebar section to collect the long moving average window, and then computed it sequentially in the main application block right after fetching the stock data dataframe:

I added this to sidebar controls:

```
ma_long = st.sidebar.slider("Long Moving Average Window", min_value=5, max_value=200, value=50, step=1)
```

I added this to main data processing block:

```
df["MA_long"] = df["Close"].rolling(window=ma_long).mean()
```

In `assignment_1.py`, I added the second slider widget to the sidebar and passed its value into a custom wrapper function decorated with `@st.cache_data`. Inside this wrapper function, I dynamically added the secondary moving average column directly onto `stock.data` after initializing the `Stock` object:

I added this to sidebar controls:

```
ma_long_win = st.sidebar.slider("Long MA Window", min_value=5, max_value=200, value=50)
```

I added this to cached helper function:

if `stock.data` is not `None` and not `stock.data.empty`:

```
stock.data["MA_long"] = stock.data["Close"].rolling(window=ma_long_win).mean()
```

Question 2: Which version was easier to change without breaking something? Describe anything that broke along the way and how you fixed it.

Working with `assignment_1.py` felt a lot safer and easier to change because all the data downloading and return logic was already tucked away inside the `Stock` class. In `demo_modified.py`, putting everything in one long script got messy quick. When I tried adding the second moving average, it messed up my Plotly chart layout and screwed with my daily change metrics. I also hit a bug in the demo where the chart showed up completely blank because of `NaN` values at the beginning of the rolling window. On the other hand, adding `MA_long` inside the `@st.cache_data` function in `assignment_1.py` kept the UI and data logic totally separate, which saved me from dealing with random chart bugs.

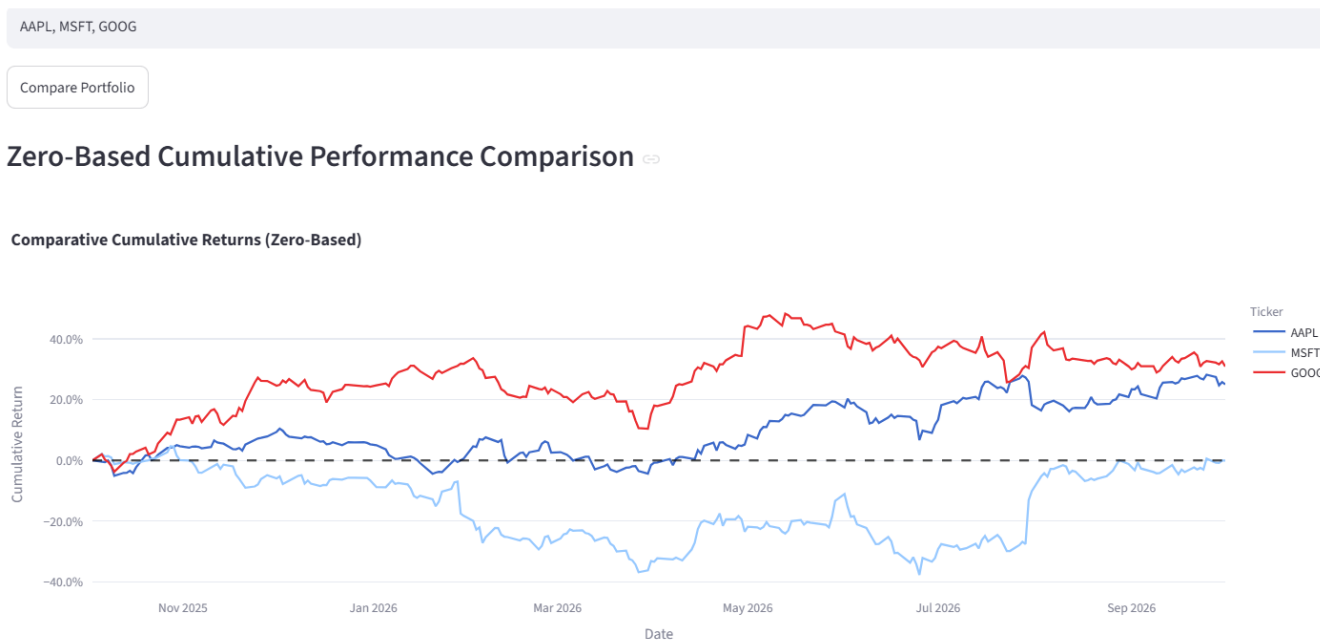
Question 3: Where did the class make the change harder, or force you to make a design decision the demo let you skip? What did you decide, and why?

The Stock class actually made things a bit trickier at first because its `__init__` method only takes one `ma_window` argument. In the demo script, I could just add as many rolling average columns to `df` as I wanted without thinking twice. But with the class, I had to make a real design decision: should I rewrite `stock.py` to support multiple moving average parameters, or just calculate the second one on `stock.data` inside `assignment_1.py`? I ended up computing `MA_long` right inside my cached function in `assignment_1.py`. This let me keep `stock.py` completely untouched while still getting the benefits of Streamlit caching.

Question 4: If someone handed you this code in six months and asked you to add another feature, which version would you rather start from? Explain using specific examples from your own code.

Hands down, I'd rather pick up the class-based code (`assignment_1.py` + `stock.py`) in six months. The demo script dumps data fetching, math, and Streamlit layout code all into one long file, so adding a new feature feels like playing Jenga with the UI. With the class structure, everything is cleanly separated. In `assignment_1.py`, plotting a chart is as simple as calling `fig_perf = stock.plot_performance()` without worrying about how the data was calculated. If someone asks me to add Bollinger Bands or RSI down the road, I can just write a quick method inside `stock.py` and keep my Streamlit frontend totally untouched.

Portfolio Stock Analysis



Comparing AAPL and GOOG over the last 6 months using the Portfolio Comparison tab reveals distinct relative performance trends. Around mid-July 2026, the zero-based cumulative performance line for AAPL crossed above GOOG, with AAPL rising toward +25% cumulative return while GOOG experienced a downward trajectory from its peak of over +40%. Leading up to this crossover, GOOG experienced several consecutive negative daily log returns while AAPL maintained steady positive daily gains. This shift in momentum allowed AAPL to sustain higher relative performance through September 2026.